

A PACKT COURSE

Beyond GraphRAG

Building an explainable financial advisor with knowledge graphs, Neo4j, and LLM agents

Alessandro Negro

Chief Scientist, GraphAware

THE PROBLEM

Good decisions need everything in one place.

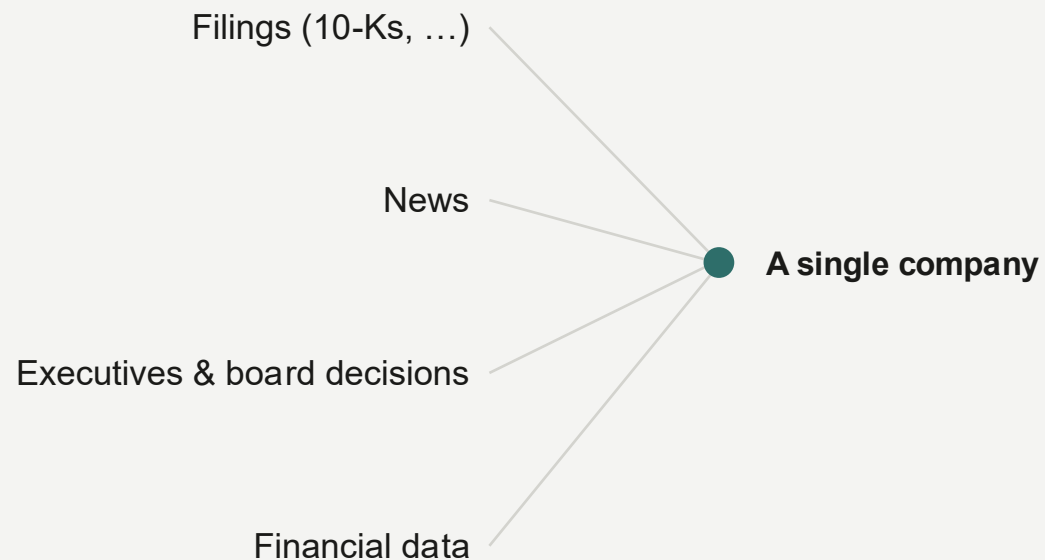
Often, the knowledge to make proper decisions is scattered across many sources in many formats. Such knowledge has to be found, aggregated, and summarized before a person can use it.



THE USE CASE

Value investing is a knowledge problem.

To judge a company, to invest properly, you must know a great deal about it — and that knowledge lives in many different places, never one.



10-K — a company's annual report filed with the U.S. SEC.

WHAT A REAL ADVISOR MUST ANSWER

Questions that quietly defeat a vanilla RAG pipeline:

What did a CEO change after joining — and how did the business change with them?

What set Company A apart from Company B in the same period — and whose approach played out better?

How did a company's stated priorities shift across consecutive annual reports?

Which executives moved between rival firms — and what happened around each move?

Multi-hop. Cross-document. Comparison. Change over time.

THE APPROACH

Gather the knowledge, organize it as a graph,
access it properly.

That spine — a knowledge graph you can grow and query — is what everything else in the course is built on.

KG (knowledge graph) — entities and the relationships between them, stored as a graph.

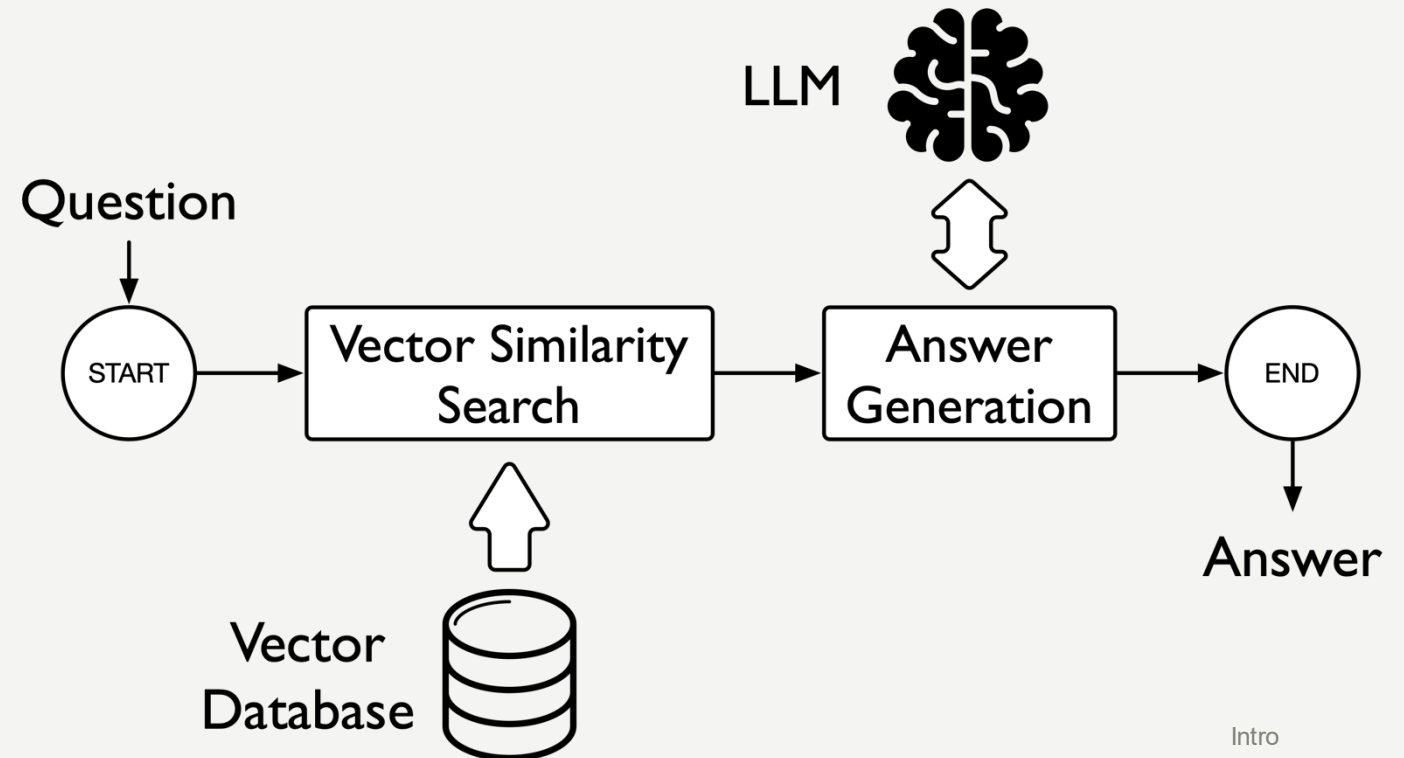
RAG stays our best friend.

The question is never whether to retrieve — it's how. RAG has many possible implementations, and the implementation is most of what decides whether it actually works for questions like these.

One-shot RAG.

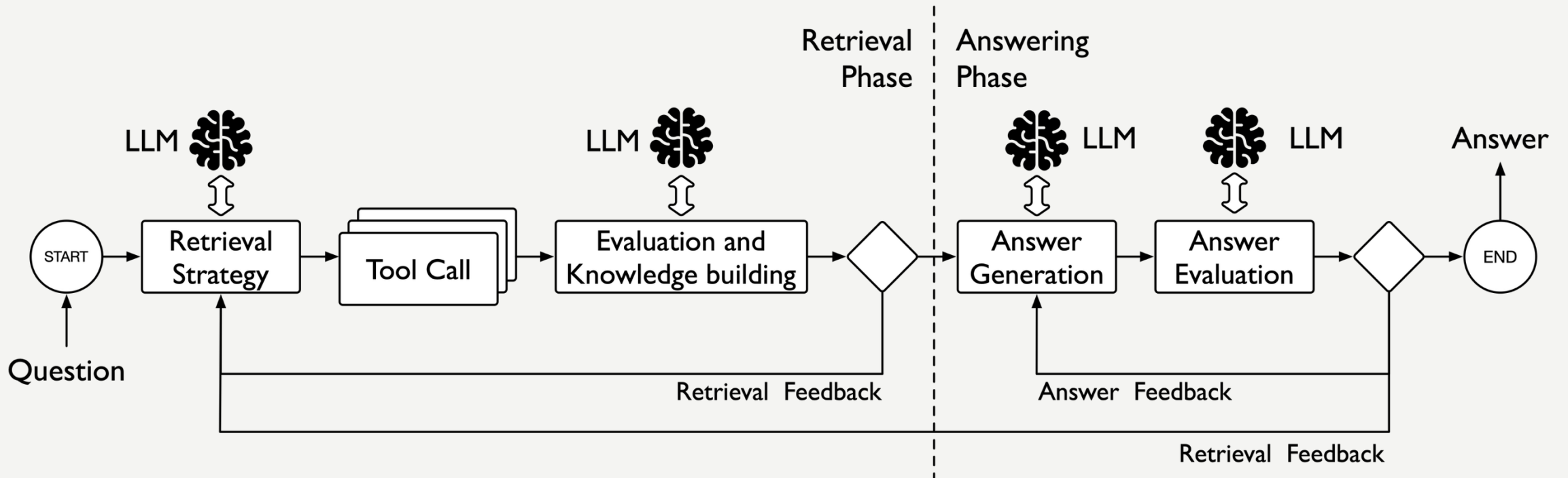
Question → Retrieve chunks → Context → Answer

It works on easy lookups — and visibly stumbles the moment a question needs more than flat similarity.



Agentic RAG.

The agent chooses a tool, uses it, grades the result — and retries when the retrieval or the answer isn't good enough.

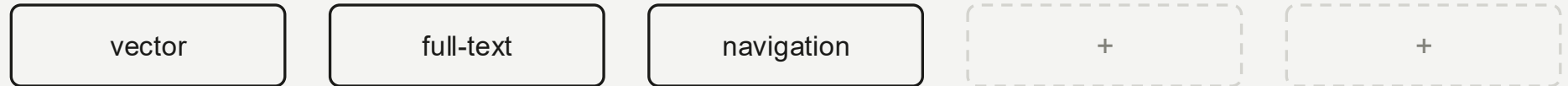


WHY THE LOOP MATTERS

One loop, many tools.

Once the agent loop exists, every later module adds a new tool to the same agent — no restructuring.

Agent



later modules add tools to the same agent

Agentic RAG is only as good as the knowledge base behind it.

A flexible agent is wasted on a store that can't grow. Extensible retrieval has to be paired with an extensible knowledge store —

a knowledge graph, not a flat vector index.

THE PATH

Seven layers. Each one earns its place.



Every module ends with a system that is visibly better than the last — motivated by a limitation you just watched the previous one hit.

Grow the graph one layer at a time — and use each layer at once.

Never build ahead of what we can demonstrate. Modules 3–7 each add one capability to the graph and one to the agent, in the same step.

THE STACK

What it's built on.

Neo4j — property graph + native vector index + text search index

LangChain — tool wrapping

LangGraph — agent loop with retrieval- and answer-grading steps, plus retry edges

Azure OpenAI — chat / agent reasoning only

Local embeddings — sentence-transformers, BAAI/bge-m3 — no embedding API call

Docling — PDF parsing & chunking

Pydantic — schema validation throughout

UP FIRST

Module 01

Baseline RAG on filings.



Ingest 10-Ks into Neo4j and stand up a one-shot RAG baseline — then find its edges.

We have a working baseline.

- 10-K filings ingested into a Company → Document → Chunk graph in Neo4j, embedded locally with bge-m3.
- One-shot vector RAG answers easy lookups end to end.

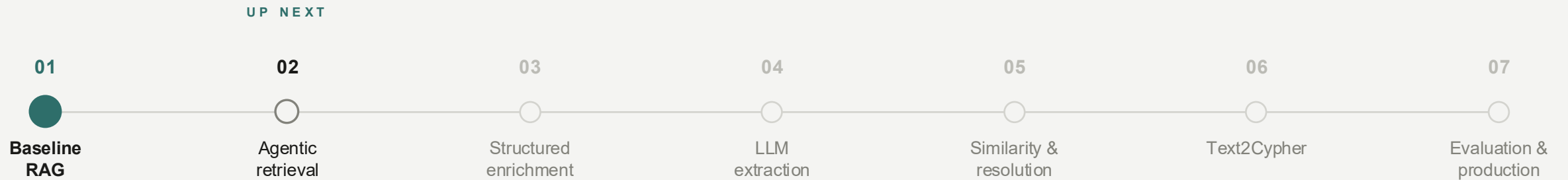
Design choice — we modeled documents as a graph from day one. The baseline's store is the substrate we'll grow, not a throwaway vector index.

But flat similarity breaks on multi-hop and cross-document questions. Next: smarter access to the same data.

RAG — retrieval-augmented generation.

WHERE WE ARE

Module 01 complete.



Up next — turn the one-shot pipeline into an agentic retrieval loop.

The pipeline is now an agent.

- One-shot retrieval replaced by a LangGraph agent with three tools: semantic search, full-text (Lucene) search, and document navigation.
- Two grading steps — one scores retrieval and can retry, one scores the final answer and can retry or go back to retrieve.

Design choice — explicit grade-and-retry edges, not a plain ReAct loop. The agent evaluates its own retrieval and its own answer.

The extensibility payoff, cashed in for the first time — but the loop is only as good as what it can reach. Time to grow the graph.

LangGraph — a framework for building agent loops as explicit graphs of steps.

WHERE WE ARE

Module 02 complete.



Up next — enrich the graph with structured external data.

The graph now holds more than text.

- New sources: Wikidata (executives, board, company profile — no API key), Sharadar (fundamentals, corporate actions), and NYT news.
- New node types — Person, Event, Article, FinancialPeriod — grown into a live graph, incrementally and tested at each step.

Every addition was checked to see it improved answers rather than just adding noise.

NYT — The New York Times.

When two sources disagree, keep the disagreement.

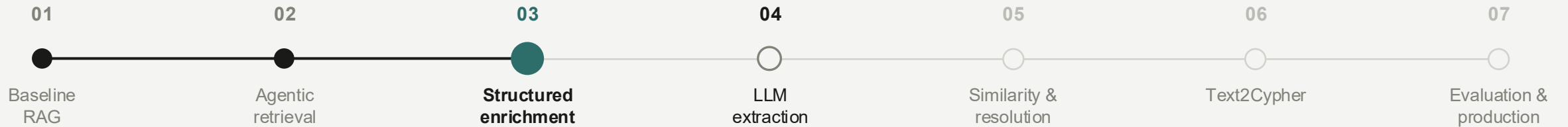
Wikidata and Sharadar sometimes classify the same company's industry or sector differently. We don't silently pick a winner — the conflict is recorded as data the agent can reason over.



WHERE WE ARE

Module 03 complete.

UP NEXT



Up next — extract entities and relationships from the text itself.

We mined structure out of the text.

Naive prompt → noisy output

Single-shot “rigorous” prompt (role, scope, predefined types) → better, still flawed

Two-phase pipeline entities → a reflection pass → relationships → what ships

Design choice — extracted entities collapse into one generic label, not dynamic per-type labels, to avoid collisions before entity resolution exists.

Real failure modes handled along the way: inconsistent outputs and hallucinated links.

LLM — large language model.

WHY IT MATTERS

Structure the agent can count on.

On a counting question, the agent without extracted structure burns retries or refuses. With it, the agent answers in fewer rounds — checked against a Cypher-computed ground truth.

Without structure

retries · refuses

With structure

correct, fewer rounds ✓

Cypher — Neo4j's query language, used here to compute the ground-truth count.

WHERE WE ARE

Module 04 complete.

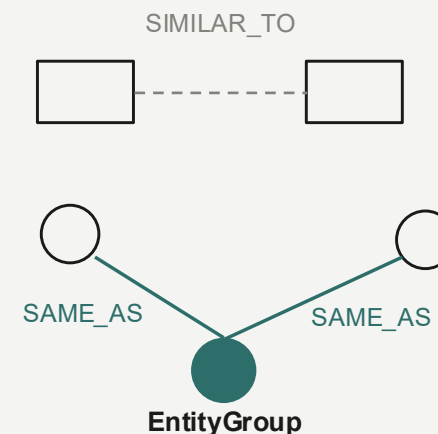


Up next — link similar content and resolve duplicate entities.

We improved the graph itself.

- SIMILAR_TO edges between chunks (vector-index search) drive redundancy-aware re-ranking and exploratory topic clustering (Louvain via GDS).
- Duplicate entities are resolved — fuzzy name matching, then an LLM judging against graph context — and merged onto a canonical EntityGroup node via SAME_AS.

This is what turns a pile of extracted mentions into a graph you can actually query.



GDS — Neo4j Graph Data Science library. Louvain — a community-detection algorithm.

WHERE WE ARE

Module 05 complete.



Up next — a text-to-Cypher escape hatch for open-ended questions.

A last-resort way to ask anything.

- A schema-aware chain translates natural language into Cypher, wired in as an agent tool with write-operation guardrails.
- Reached for only when the curated tools can't answer — powerful, and the one tool we trust least by default.

Curated tools first; generation is the escape hatch, not the default.

Cypher — Neo4j's query language. NL — natural language.

Powerful, and not yet fully trustworthy.

Building it surfaced a real correctness bug — we found a measured mitigation, but generation stays probabilistic, improved, not guaranteed. And a measured limit: single-hop generated queries are reliable, multi-hop joins are not yet. We present both as genuine limitations, not footnotes.

This is the *explainable* half of the title doing its work.

WHERE WE ARE

Module 06 complete.



Up next — evaluation, and what production actually demands.

We measured whether any of it worked.

- LLM-as-judge scores correctness, completeness, and grounding — multi-part questions decomposed into independently scored sub-claims.
- Bias fixes: an anchored 1/3/5 rubric, and rationale written before the score — so the number follows the reasoning, not the reverse.

Evaluation ran through the whole build — “did this layer actually help?” — and is consolidated here.

LLM-as-judge — using an LLM to grade another system's answers against explicit criteria.

NOW WHAT

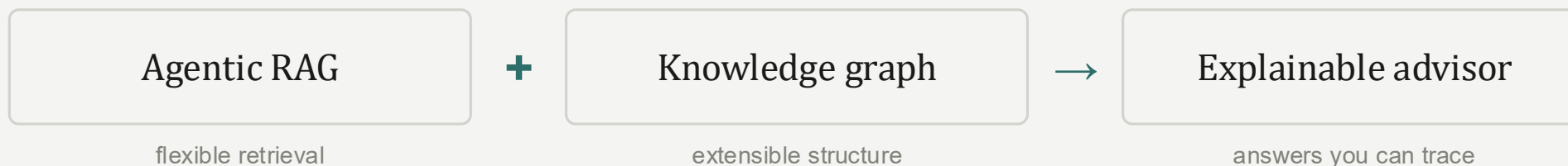
Running it for real.

Monitoring · Graph refresh · Cost · Governance

The considerations that separate a course demo from a financial decision-support system in production.

A flexible agent over an extensible graph.

Agentic RAG gave us flexibility in how we retrieve — choose a tool, use it, grade the result, retry. The knowledge graph gave us expansibility in what there is to retrieve — every module added new structure, and the same agent used it the moment it landed.



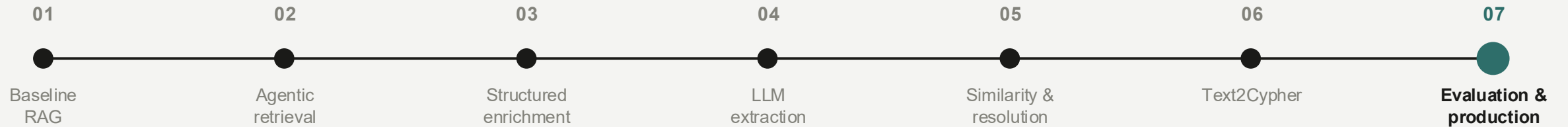
The questions that defeated plain RAG — now in reach.

Each hard question needed a different capability, and each got one: a tool to ask with, and the graph structure to answer from.

- Multi-hop — traverse people, roles, and companies, not just similar text.
- Cross-document & cross-company — links that live in the graph, discovered once and reused.
- Comparison — structured financials and events, side by side.
- Change over time — filings and periods anchored so shifts are queryable.

THE PATH, COMPLETE

Seven layers, built and used.



An explainable financial advisor, grown one layer at a time.

Alessandro Negro · Chief Scientist, GraphAware

Thank You!